Kapitel 2 Grundlagen

- ## Terminierung

Kryptographie: Lehre der Absicherung von Nachrichten durch Verschlüsseln
Kryptanalyse: Kunst Chiffretext aufzubrechen, ohne Kenntnis des Schlüssels
Kryptologie: vereinigt Kryptographie und Kryptanalyse
Steganographie: geheime Nachrichten werden nicht verschlüsselt, sondern versteckt

Alphabet A: endliche Menge von Zeichen. n=|A| ist die Mächtigkeit
Klartext: lesbarer Text einer Nachricht; wird mit M bezeichnet. Zeichenkette über dem Alphabet A
Geheimtext/Chiffretext: Zeichenkette über dem gleichen Alphabet A
Schlüssel: Zeichenketten

Verschlüsselung: Verfahren um Nachrichten unverständlich zu machen
Chiffre E: invertierbare (umkehrbare) Abbildung welche aus M und K den Geheimtext C erzeugt. Voraussetzung für die Invertierbarkeit ist die Injektivität.
Entschlüsselung: Umkehrung von E zur Wiederherstellung von M wird mit D bezeichnet

- E(M) = C
  D(C) = M
  D(E(M)) = M

Alle kryptographischen Verfahren haben die Aufgabe eine der 4 folgenden Eigenschaften zu erfüllen:

- **Geheimhaltung**: Lesen einer Nachricht unmöglich, bzw. schwierig zu machen
- **Authentifizierung**: Empfänger kann sich sicher sein, dass die Nachricht nicht von jemand anderem gesendet wurde
- **Integrität**: Nachricht darf während der Übermittlung nicht verändert werden
- **Verbindlichkeit**: Sender kann nicht leugnen die Nachricht abgeschickt zu haben

- **Kryptographische Algorithmen**
  → Mathematische Funktion zur Ver- und Entschlüsselung

  - **Symmetrische Algorithmen**: zum Chiffrieren und Dechiffrieren wird immer der gleiche Schlüssel K benutzt

    - $E_K(M) = C$
      $D_K(C) = M$
      $D_K(E_K(M)) = M$

  - **Assymetrische Algorithmen**: zum Chiffrieren wird Schlüssel K_1 benutzt und zum Dechiffrieren ein anderer Schlüssel K_2

    - $E_{K1}(M) = C$
      $D_{K2}(C) = M$
      $D_{K2}(E_{K1}(M)) = C$

1. Man unterscheidet zwischen Strom- und Blockchiffren
   - **Stromchiffren**: ein Zeichen nach dem anderen
   - **Blockchiffren**: Nachricht wird in Blöcke zerteilt und ein Block nach dem anderen verschlüsselt

2. **Kryptosystem**: Vereinigung von Algorithmus, zugehörigem Schlüssel und den verschlüsselten Nachrichten.

3. **Eingeschränkter Algorithmus**: Sicherheit hängt davon ab, dass der Algorithmus geheim ist.

   Problem: Verlässt eine Person die Firma, muss Algorithmus verändert werden!

   Heutzutage: Algorithmus ist bekannt!!!!

   Kerkhoff: Sicherheit eines Verschlüsselungsverfahrens darf nur von der Geheimhaltung des Schlüssels abhängen und noch von der Geheimhaltung des Algorithmus.

   Frühzeitige Offenlegung des Algorithmus -> größere Sicherheit
   Experten können Algorithmus testen
   => wichtiger Bestandteil der starken Kryptographie

- **Public-Key-Algorithmen**

  ↳ Sichere Kommunikation im Internet

  Systeme wie PGP zum Verschlüsseln von E-Mails undenkbar

  Möglichkeiten:
  - Nachrichten austauschen
  - Dokumente signieren
  - …

  Algorithmen mit öffentlichen Schlüsseln sind asymmetrisch

  ➔ einen geheimen Schlüssel S und einen öffentlichen Schlüssel P

  Beispiel: Bob will geheime Botschaft senden -> erzeugt P_B und lässt ihn allen Kommunikationspartnern zukommen und S_B den er für sich behält -> Alice benutzt P_B zum Verschlüsseln -> Bob entschlüsselt mit S_B

  - $E_{PB}(M) = C$
    $D_{SB}(C) = M$
    $D_{SB}(E_{PB}(M)) = M$

  Signieren geht genau andersherum!!

- **Kryptanalyse**

  - **vollständiger Aufbruch**: Schlüssel K wird gefunden und mit $D_K(C) = M$ kann alles entschlüsselt werden
  - **globale Deduktion:** Finden eines zu D_K äquivalenten Algorithmus ohne Kenntnis des Schlüssels
  - **lokale Deduktion**: Finden des Klartextes für nur einen abgefangen Chiffretext
  - **Informationsdeduktion**: nur eingeschränkte Informationen über den Schlüssel oder Klartext

Verschiedene Arten von Kryptanalytischen Angriffen auf ein Kryptosystem

- **Ciphertext-Only-Angriff**: nur bestimmte Menge Chiffretext
- **Known-Plaintext-Angriff**: der zugehörige Klartext zum Chiffretext ist bekannt
- **Chosen-Plaintext-Angriff**: beliebigen Klartext vorgeben, Möglichkeit zu diesem vorgegebenen Text an den Chiffretext zu gelangen
  -> Knacken von Publick-Key-Systemen
- **Chosen-Ciphertext-Angriff**: beliebiger Chiffretext vorgegeben, Möglichkeit um an zugehörigen Klartext zu gelangen
- **Angriff mit Gewalt**: Person mit Zugang zum Schlüssel wird bedroht
- **Angriff mit gekauftem Schlüssel**: Schlüssel wird „gekauft“

Es gilt: Bei den Verfahren der starken Kryptographie stellt meist der Mensch als Besitzer des Schlüssels die größte Sicherheitslücke dar.

- **Brute-Force-Angriff**: Angriff bei dem alle Schlüssel ausprobiert werden
  - **Ciphertext-Only-Angriff**: Solange probieren bis der berechnete Klartext Sinn macht
  - **Known-Plaintext-Angriff**: Solange Verschlüsseln der Klartexte bis er mit Chiffretext übereinstimmt

=> Es lassen sich fast alle Kryptosysteme, mit entsprechendem Aufwand, kancken.

ABER: Algorithmus gilt als sicher, wenn:

- Geldaufwand zum Knacken übersteigt Wert der Daten
- Zeit zum Knacken höher als die Zeit, die die Daten geheim bleiben müssen
- Das mit einen Schlüssel chiffrierte Datenvolumen kleiner ist, als die Datenmenge zum Knacken

  - **Uneingeschränkt sicher**: Klartext kann nicht ermittelt werden, auch wenn Chiffretext in beliebigem Umfang vorhanden ist

Berechnung des Aufwands:

- **Berechnung Komplexität**: Rechenzeit in Abhängigkeit von der Schlüssellänge
- **Speicherplatzbedarf**
- **Datenkomplexität**: Menge der benötigten Eingabedaten

- ## Sicherheit von Schlüsseln

56-Bit langer Schlüssel (DES)

-> Schlüsselraum: $2^{56} \approx 7*10^{16}$

$9*10^{10}$ Schlüssel pro Sekunde können getestet werden

-> mittlere Zeit von $3{,}9*10^{5}$ Sekunden ≈ 4,5 Tage

=> DES ist nicht mehr sicher

100-Bit langer Schlüssel

-> Schlüsselraum: $2^{100} \approx 1{,}3*10^{30}$ Schlüssel

-> mittlere Zeit von $7*10^{18}$ Sekunden ≈ $2*10^{11}$ Jahre

=> Schlüsselraum sollte möglichst groß sein, mindestens so groß, dass der Aufwand für einen Angriff unakzeptabel hoch ist.

Kapitel 3 Klassische Chiffren

- **Transpositionschiffre**: Permutation der Klartextzeichen. Zeichen bleiben gleich, tauschen aber ihr Plätze
- **Substitutionschiffre**: Jedes Zeichen des Klartextes wird durch ein anderes ersetzt, Position bleibt gleich.
  - **Monoalphabetisch**: Klartextzeichen immer aufs gleiche Geheimtextzeichen
  - **Polyalphabetisch**: nicht monoalphabetisch

Es gibt 26! ≈ $4*10^{26}$ monoalphabetische Chiffren über dem Alphabet
-> leicht zu knacken: Verschlüsseln von Binärdaten gibt es nur 2 monoalphabetische Chiffren (Identität und Negation)

- **Verschiebechiffre** (Substitutionschiffre): Klartextzeichen z wird durch ein um k-Zeichen im Alphabet verschobenes Zeichen ersetzt.
  Alphabet mit n-Zeichen: Zeichen durchnummeriert von 0 bis n-1
  => z -> (z+k) mod n

  => **leicht zu knacken**: alle Verschiebungen durchprobieren bis lesbarer Text entsteht. Wenn Alphabet zu groß, statistische Analyse. Häufigkeit von Buchstaben im deutschen Alphabet variiert kaum. Häufigkeit im Geheimtext wird mit der mittleren Häufigkeit verglichen.

- **Multiplikative Chiffren**: jedes Klartextzeichen z wird mit einer Zahl t {0,…,n} multipliziert.
  t und n = |A| teilerfremd. Es gilt ggT(t,n) = 1
  => z -> (z*t) mod n

  Zu jeder multiplikativen Chiffre E mit ggt(t,n) = 1 gibt es eine multiplikative Dechiffrierfunktion D mit D(E(z)) = z
  => D: z‘ -> bz‘ mod n
  => D(E(z)) = (b(zt)mod n)mod n = ((bt)mod n*z)mod n
  = (1z)mod n = z

  Schlüsselraum ist ggtT(t,26) = 1 und t < 26
  => 12

- **Tauschchiffren** (Affine Chiffren): Eine Chiffre E mit z -> (z*t+k) mod n
  Es wird ggT(t,n) = 1 für die Invertierbarkeit vorausgesetzt
  Suchen eienr multiplikative Inversen b zu t
  Für b muss gelten t*b Ξ 1 mod 26

- **Polyalphabetische Chiffre**: Jedes Klartextzeichen kann je nach Kontext auf unterschiedliche
  Chiffretextzeichen abgebildet werden.

  - **Homophone Chiffren**: jedes Zeichen kommt im Geheimtext gleich häufig vor.
    -> Homophone Chiffren verschleiern die Häufigkeit der Klartextzeichen
    ABER: Angriffe möglich -> statistische Häufigkeit von Paaren

- **Vigenere Chiffren**: basiert auf der Verschiebechiffre. Jedes Zeichen im Alphabet wird verschoben
  -> Abhängig von der Position des Zeichens im Text und dem Schlüsselwort

  - **Algorithmus**: Schlüsselwort wird vereinbart. Es wird das Vigenere-Quadrat zum Ver- und Entschlüsseln benutzt
    => K ist das Schlüsselwort der Länge k
    V die Matrix des Vigenere-Quadrats und z die Nummer des zu codierenden Zeichens im Alphabet, sowie i die Position von z im Klartext
    => $E(z,i) = (z+K_{i \bmod k}) \bmod n = V_z K_{i \bmod k}$ (n = Länge des Alphabets)

  - **Kryptanalyse**: wichtigstes Ziel ist die Bestimmung der Schlüsselwortlänge k, wenn k gefunden ist funktioniert es wie bei der Verschiebechiffre, wobei k Geheimtexte separat analysiert werden. Text 1 entsteht aus der Teilfolge all der Zeichen, die mit dem ersten Schlüsselwortzeichen verschlüsselt wurden, Text i die mit dem i-ten. Matrix wird erstellt
    2 Test zur Ermittlung der Schlüssellänge: Kasiski und Friedman
    => Kasiski liefert Kandidaten bis auf ein Vielfaches.
    Friedmann liefert ungefähren Wert für die Größenordnung
    Zusammen führen sie meist zum Ziel

    - **Kasiski-Test**: Geheimtext nach Wiederholungen von Zeichenfolgen mit mindestens 3 Zeichen und misst den Abstand. Je länger die Zeichenfolgen sind, desto größer die Wahrscheinlichkeit, das Abstand ein Vielfaches der Schlüssellänge
      => Geheimtext muss viel länger sein, als das Schlüsselwort

    - **Friedmann-Test**: Buchstaben innerhalb der Teiltexte sind nicht zufällig
      verteilt. Es gibt innerhalb der Teiltexte mehr Wiederholungen von Buchstaben als zwischen den Teiltexten

      Koinzidenzindex I berechnen: Wahrscheinlichkeit, dass 2 zufällig gewählte Buchstaben mit beliebigem Abstand gleich sind

- **One-Time-Pad**: Vigenere-Chiffre mit unendlich langem Schlüsselwort

  => Jeder einzelne Schlüsselwortbuchstabe wird genau einmal in höchstens einer Nachricht benutzt

  binär verschlüsselt {0,1}

  Klartextbit z wird mit Zufallsbit r verschlüsselt:

  Z -> (z+r) mod 2 = z XOR r = z(+)r

  => Klasse der Stromchiffren

  Wegen z(+)r(+)r = z Dechiffrieren mit gleichem Schlüssel und Funktion wie Chiffrieren

  => uneingeschränkt sicher

  Chiffriersystem perfekt, wenn bei beliebigem Klartext und Chiffretext C die a-priori-Wahrscheinlichkeit gleich der bedingten Wahrscheinlichket.

  Es gibt keine Möglichkeit, aus einem beliebig langem Chiffretext den Klartext zu rekonstruieren

- **Zusammenfassung**: statistische Verfahren genügen zum Aufbrechen monoalphabetischer Chiffren. One-Time-Pad ist sicher, jedoch keine praktische Bedeutung im Vergleich zu den modernen Blockchiffren, die mit 128-bit auskommen, da der Aufwand zu hoch ist.

Kapitel 4 Moderne Blockchiffren

Bildung des Geheimtexts durch Konfusion und Diffusion

Konfusion: Mischen, statistische Eigenschaften des Chiffrextes möglichst Folge aus Zufallszahlen

Diffusion: jedes Klartextbit wird auf viele Chiffretextbits verteilt

Hohe Sicherheit -> großer Schlüsselraum

Nicht möglich, die Gesamtzahl aller Chiffren mit Blocklänge 64-Bit: 2^{64}!

Zum codieren wird also ein Schlüssel der Länge $\log_2 (2^{64}!) \approx 2^{70} \approx 10^{21}$ Bit benötigt (ungefähr 10^{11} GByte)

=> kleine Teilmenge von Blockchiffren mit kurzem Schlüssel

- **DES**: symmetrische Blockchiffre

  LUCIFER: arbeitet abwechselnd mit Substitution und Permutation auf den Blöcken und wiederholt diesen Prozess mehrfach mit unterschiedlichen Teilen des Schlüssels

  - **Übersicht**: Klartext wird durch die Eingangspermutation gemischt, durchläuft dann die 16 Runden und nach der Schlusspermutation ist der Chiffretext erzeugt. Schlusspermutation ist invers zur Eingangspermutation

  - **Eine Runde**: 64-Bit Block wird in die linke Hälfte L_0 und die rechte Hälfte R_0 mit je 32-Bit geteilt. In jeder Runde wird die rechte Hälfte unverändert als Linke Hälfte der nächsten Runde übernommen. $L_i = R_{i-1}$. Auf Teilschlüssel K_i und die rechte Hälfte R_{i-1} wird die Funktion f angewendet. Das Ergebnis wird mit linker Hälfte L_{i-1} XOR-verknüpft und als rechte Hälfte der nächsten Runde übernommen. Funktion f: S-Box-Transformation. Das jeweils erste und letzte Bit eines Blocks, als Zahl, bestimmt dieZeile in der zugehörigen S-Box. Die Bits 2 bis 5 werden zur Auswahl der Spalte benutzt.

  - **Die 16 Teilsschlüssel**: Der gleiche Algorithmus kann auch zur Entschlüsselung benutzt werden. Teilschlüssel müssen in umgekehrter Reihenfolge angewendet werden. Schlüsselverschiebungen erfolgen nach rechts statt nach links.

  - **Sicherheit und Nichtlinearität**: Sicherheit basiert auf der Konstruktion der S-Boxen. Minimale Änderung des Klartextes -> dramatische Änderung des Geheimtextes -> Lawineneffekt

  - **Triple DES**: $C = E_{K1}(D_{K2}(E_{K1}(M)))$ -> DES dreimal, mit 2 verschiedenen Schlüsseln

    Warum nicht nur 2-mal?

    => anfällig gegen Meet-in-the-Middle Angriff

    => nur doppelt so aufwendig wie bei einfachem DES

- **AES**: Nachfolger von DES

  Anforderungen: **Formal**: symmetrische Blockchiffren. Blockgröße 128-Bit;
  Schlüssellänge 128 bis 256 Bit
  Sicherheit: Angriffe aller Art
  Einfachheit: Design
  Flexibilität: auch andere Blockgrößen und Schlüssellängen
  Effizienz: effizienter als Triple-DES
  Implementierung: einfach in Hard- und Software

  - **Blockchiffre Rijndael**: iterierte Blockchiffre deren Blocklänge b und Schlüssellänge k unabhängig auf einen der Werte 128, 192 und 256-Bit gesetzt werden können. Zahl der Runden r variiert, abhängig von diesen Werten, zwischen 10 und 14. Zwischenergebnisse des Verschlüsselungsprozesses werden Zustand S_i genannt.

- **Betriebsmodi von Blockchiffren**: einfachste Art ist das Verschlüsseln eines Blocks nach dem anderen mit dem gleichen Schlüssel. Gleiche Blöcke werden gleich verschlüsselt. Für festen Schlüssel lässt sich eine Tabelle mit aller Klartext-Chiffretext-Paare anlegen -> electronic code book (ECB)
  => **Lösung**: CBC-Modus (cipher block chaining). Für alle Blöcke ab dem zweiten wird vor Verschlüsselung der Chiffretext des Vorgängerblocks mit dem aktuellen Klartextblock XOR-verknüpft und dann verschlüsselt. Jedoch werden auch im CBC-Modus gleiche Klartexte identisch verschlüsselt.
  => **Lösung**: Initialisierungsvektor als ersten Block verwenden
  Was passiert mit letztem evtl. nicht ganz vollen Block?
  => **padding**: Rest des letzten Blocks wird nicht Zufallsbits aufgefüllt.

Kapitel 5 Public-Key-Kryptographie

AES bietet keine Lösung für das Schlüsseltauschproblem
=>Wie tauscht Alice den Schlüssel über einen unsicheren Kanal aus?
Lösung: Erfindung der Public-Key-Kryptographie
=>RSA-Algorithmus, kann zum Verschlüsseln und Signieren benutzt werden

- ## RSA-Algorithmus

  - **Der Algorithmus: besteht aus 2 Teilen: Chiffrieralgorithmus und vorallem** Schlüsselerzeugung
    1. Wähle zufällig zwei große Primzahlen p und q. Länge der Zahlen sollte mindestens 512-Bit sein
    2. Berechne n=p*q (n hat Länge von mindestens 1024-Bit)
    3. Wähle eine kleine ungerade natürliche Zahl e, die zu $\phi(n)=(p-1)*(q-1)$ relativ prim ist, d.h. es gilt $ggT(e,\phi(n))=1$
    4. Berechne d als Lösung der Gleichung $e*d \bmod \phi(n)$ (=>$d=e^{-1} \bmod \phi(n)$
       d kann mit dem erweiterten Euklidischen Algorithmus
    5. Paar P=(e,n) bekannt geben als öffentlicher Schlüssel
    6. Paar S=(d,n) geheim halten als geheimen Schlüssel

  - **Anwendung von RSA**

    Verschlüsseln: Nachricht M wird codiert durch $E(M)=M^e \bmod n$
    Entschlüsseln: Chiffretext wird decodiert durch $D(C)=C^d \bmod n$

    Beispiel: p und q wählen -> n=p*q -> $\phi(n)=(p-1)*(q-1)$ -> e wählen und mit erweitertem Euklidischen Algorithmus $d=e^{-1} \bmod \phi(n)$ berechnen
    P und q vernichten und e und n bekanntgeben
    Nachricht M in Blöcke zerlegen, die kleiner als n sind

    Korrektheit: E und D sind invers zueinander: D(E(M)) = E(D(M)) = M

  - **Sicherheit von RSA**: Sicherheit von RSA basiert auf der Schwierigkeit große Zahlen in ihre Primfaktoren zu zerlegen.
    Für die Sicherheit von RSA ist es sehr wichtig, dass die beiden gewählten Primzahlen p und q nicht öffentlich bekannt sind, bzw. dass sie schwer zu erraten sind. P und q müssen effizient und möglichst zufällig aus einer möglichst großen Menge von Primzahlen gewählt werden.

- **Effiziente Primzahltests:** Primzahltest von Miller und Rabin:

  =>n prim: $a^{(n-1)/2}$ = +-1 für alle a ϵ $Z_n\backslash\{0\}$

  =>n nicht prim: $a^{(n-1)/2}$ = +-1 für höchstens die Hälfte der a ϵ $Z_n\backslash\{0\}$

  Es ist auch wichtig, dass es genügend viele Primzahlen mit einer vorgegebenen festen Länge gilt.

  Fermat:

  a^{n-1} mod n = 1

  2^{111} mod 112 = $2^{(64+32+8+4+2+1)}$ mod 112

  $=(((((2^2)^2)^2)^2)^2 * (((2^2)^2)^2)^2 * (2^2)^2 * 2^2 * 2 * 1)$ mod 112

- **Effizienz und Implentierung von RSA**: Berechnung von M^e mod n zum Beispiel durch schrittweise Multiplikation von M mit sich selbst erfolgen

  => extrem zeitaufwändig, daher wiederholtes Quadrieren (Fermat)

- **Angriffe gegen Public-Key-Verfahren**: Public-Key-Verfahren besitzen eine deutliche Schwäche. Es kann ganz einfach ein Chosen-Plaintext-Angriff durchgeführt werden, denn mit dem öffentlichen Schlüssel ist es leicht, sich für beliebige Mengen von Klartext den zugehörigen Chiffretext zu erzeugen

  - **Chosen-Ciphertext-Angriff mit Social Engineering**: Angriff ist erfolgreich, wenn die Reihenfolge beim Verschlüsseln keine Rolle spielt (RSA)

  - **Angriffe aufgrund von Seiteneffekten**: Angreifer misst die Laufzeiten von MODULAR-EXPONENT.

    Lösung: - MODULAR-EXPONENT so verändern, dass Laufzeit konstant gleich der maximalen Laufzeit wird.

    - Warteschleife bei jedem Anruf von MODULAR-EXPONENT. Dies ist jedoch nicht ganz sicher, da Rauschen

    - Gegner blenden, indem man den Chiffretext vor dem Entschlüsseln mit einer Zufallszahl r multipliziert.

      1. Erzeuge Zufallszahl r zwischen 0 und n-1
      2. Berechne c‘ = cr^e mod n, wobei e der öffentliche Exponent ist
      3. Berechne m‘ = $(c')^d$ mod n mit MODULAR-EXPONENT
      4. Berechne m = $m'r^{-1}$ mod n

         => Damit ist die Laufzeit für die Dechiffrierung unabhängig vom Chiffretext c und es gibt keine Möglichkeit, den Schlüssel zu bestimmen

- **Schlüsseltausch**: Kommunizieren 2 Parteien mit geheimen Schlüssel, d.h. mit einem symmetrischen Verfahren, so wird die Kommunikation umso sicherer, je öfter der Schlüssel gewechselt wird.

    - **Schlüsseltausch mit symmetrischen Verfahren**: Eine dritte Partei, das Trustcenter zur Mithilfe herangezogen. Jede Partei muss mit dem Trustcenter auf irgendeinem Weg einen eigenen geheimen Schlüssel vereinbart haben.

    - **Man-in-the-Middle-Angriff**: Angreifer trennt Verbindung zwischen Alice und Bob in der Mitte auf und täuscht Alice vor, Bob zu sein und umgekehrt. Beim Angriff auf Public-Key-Algorithmen kann der Angreifer so alle Nachrichte mithören und verändert weiterleiten, ohne Algorithmus anzugreifen oder zu knacken. Mallory fängt bei einem Schlüsseltausch mit einem Public-Key-Algorithmus die beiden öffentlichen Schlüssel P_A und P_B ab und leitet jeweils seinen eigenen öffentlichen Schlüssel P_M weiter. Ist immer dann möglich, wenn Alice und Bob keine Möglichkeit haben zu überprüfen mit wem sie tatsächlich kommunizieren.

        => Bei der Verwendung eines fremden öffentlichen Schlüssels muss möglichst immer die Authentizität de Schlüssels geprüft bzw. sichergestellt werden.

Kapitel 6 Authentifikation und digitale Signatur

- **Einwegfunktion und Einweg-Hash-Funktion**: Das zu signierende Dokument wird mit dem geheimen Schlüssel des Unterzeichners verschlüsselt.

  => **Problem**: Größe wächst linear zur Dokumentengröße

  => **Lösung**: Es wird nur der Hash-Wert verschlüsselt, welcher mit einer Einwegfunktion berechnet wird.

  Einwegfunktion: Funktionen f, die sich leicht berechnen lassen, deren Umkehrung f^{-1} jedoch nicht, insbesondere wenn die Funktion f öffentlich ist.

  Beispiel 1: Nullstellenbestimmung bei Polynomen.

  Beispiel 2: RSA-Verschlüsselung M^e mod n ist einfach, die Umkehrung sehr schwierig

  Einweg-Hash-Funktion: Einwegfunktionen, die beliebig lange Klartexte auf einen Hash-Wert fester Länge abgebildet.

  => Abbildung ist nicht injektiv. Da Hash-Wert meist kürzer als Klartext

  -> Auf einen Hash-Wert werden viele Klartexte abgebildet.

  Lösung: Es muss schwierig sein, zwei Klartexte mit dem gleichen Hash-Wert zu finden.

  Zur Berechnung des Hash-Wertes werden nacheinander die Teile M_i der Eingabe als Schlüssel für AES verwendet. Wichtige Einweg-Hash-Funktionen sind:

  1. **MD4**: Hash-Wert ist 128-bit lang
  2. **MD5**: Verbesserung von MD4 mit 128-Bit Hash-Wert. Wird in PGP verwendet
  3. **SHA**: Verbesserung von MD4. SHA-1 mit 160.Bit
  4. **RIPE-MD**: Variante von MD4 mit 160-Bit langem Hash-Wert. Sehr sicher.

- **Passwortverschlüsselung**: Hierbei werden Einwegfunktionen verwendet, und Passwortverschlüsselung zur Benutzerauthentifikation auf Rechnern.

  => **Wörterbuchangriffe**: Angreifer stellt Liste der 100000 gängigsten Passwörter auf, wendet die (bekannte) Einwegfunktion an und speichert die Ergebnisse zusammen mit den Passwörtern. Angreifer stiehlt Passwortdatei und vergleicht diese mit seinen Daten.

  => **Lösung**: **Salt**: Passwort wird mit Zufallssequenz verknüpft, verschlüsselt und zusammen mit Zufallssequenz abgespeichert.

- **Der Geburtstagsangriff**: Angreifer tauscht das Dokument gegen ein anderes aus, das jedoch den gleichen Hash-Wert liefert. Angreifer ist Vertragspartner, erstellt aber so lange unterschiedliche Versionen eines Dokumentes, bis 2 Versionen den gleichen Hash-Wert liefern.

  => Hash-Wert muss doppelt so lang gewählt werden wie für einfache Angriffe. Heutzutage werden Längen von 128 bis 160 Bit verwendet.

- **Zero-Knowledge-Protokolle**: Benutzerauthentifikation noch nicht sicher, da Passwort im Klartext eingetippt wird und zum Server gesendet wird.
  => Protokolle, die ohne Übertragen des Schlüssels im Klartext arbeiten

  - **Challenge-and-Response**: Vorteil ist, dass kein Schlüssel übertragen wird und dass bei jeder neuen Authentifikation andere Daten übertragen werden.
    Der Nachteil ist, dass der Server in seiner Datenbank die Schlüssel (Passwörter) im Klartext abspeichern muss

  - **Die Idee der Zero-Knowledge-Protokolle**: werden dazu benutzt, jemanden davon zu überzeugen, dass man ein Geheimnis kennt, ohne jedoch auch nur ein Bit an Informationen über das Geheimnis preiszugeben

- **Digitale Signaturen:**
  1. **Authentisch**: Unterzeichner hat willentlich unterschrieben
  2. **Fälschungssicher**: Unterzeichner und kein anderer hat das Dokument unterschrieben
  3. **Nicht wiederverwendbar**: Unterschrift kann nicht auf ein anderes Objekt kopiert werden
  4. **Unveränderbar**: Dokument kann nach der Unterzeichnung nicht mehr geändert werden
  5. **Bindend**: Unterzeichner kann nicht behaupten, dass er es nicht unterzeichnet hat

  Unterschriften mit Tinte auf Papier erfüllen keine dieser Aussagen vollständig. Digitale Signaturen erfüllen alle bis auf die erste Aussage mit sehr hoher Sicherheit.
  Es wird nicht das komplette Dokument signiert, sondern nur der mittels einer öffentlichen bekannten Einweg-Hash-Funktion f berechnete Hash-Wert.
  => **Signieren**: $(M, E_{SA}(f(M)))$
  => **Prüfen**: $D_{PA}(E_{SA}(f(M)))$ und $f(M)$ und vergleicht die beiden Werte

  - **Digital Signature Algorithm (DSA):** Eignet sich nicht zum Verschlüsseln. DSA wurde zum neuen Digital Signature Standard (DSS) für das Signieren von amerikanischen Regierungsdokumenten. Wird in PGP ab Version 5 eingesetzt.

  - **Blinde Signaturen**: Unterzeichner sieht das Dokument beim Unterschreiben nicht. Alice will Dokument M von Bob blind signieren lassen.
    => verpackt M und lässt es von Bob signieren und packt es dann wieder aus (entschlüsselt es), so dass die Unterschrift erhalten bleibt.

- **Digitale Signaturen in der Praxis**: Alice will eine E-Mail signieren. Sie klickt die Schaltfläche „signieren“ an. Sie wird aufgefordert ihr Passwort (passphrase) einzugeben. „passphrase“ ist ein möglichst langes Passwort, mit dem ihr geheimer Schlüssel, symmetrisch verschlüsselt, gespeichert ist. Schlüssel wird dechiffriert und die E-Mail wird signiert. Dies wird bei PGP verwendet. Überprüft wird die Signatur mit der Schaltfläche „Signatur verifizieren“ beim Empfänger. Hierfür benötigt er den öffentlichen Schlüssel von Alice.

    - **Speichern des geheimen Schlüssels**: Er ist untrennbar mit der Identität des Besitzers verbunden. Wird er gestohlen besitzt der Dieb die Identität des Opfers. Passwort wird meist auf der Festplatte gespeichert, deshalb muss es verschlüsselt werden.

        => Das Passwort (passphrase) für die Verschlüsselung des geheimen Schlüssels eines Public-Key-Verfahrens, muss mindestens 80 Zeichen lang sein.

    - **Vertrauen in die Software**: Anwender sollte bei der Installation und vor jedem Start eines Kryptographie-Programms die Signatur des Herstellers kontrollieren.

- **Authentifikation mit digitaler Signatur**: Alice signiert mit S_A eine Zufallszahl r, die ihr Bob zuschickt und Bob prüft die Signatur mit P_A.
  Wegen der Verwendung eines Public-Key-Verfahrens ist Alice die einzige Person mit Kenntnis ihres geheimen Schlüssels.

- **Message-Authentication-Code (MAC**): Unterschied zu digitalen Signatur besteht in der Verwendung einer symmetrischen Chiffre statt eines Public-Key-Algorithmus. Der Schlüssel K muss beiden Partnern bekannt sein, und ist somit kein Ersatz für die Digitale Signatur. Er kann nur die Authentizität der Nachricht sicherstellen, nicht jedoch die des Absenders.

- **Biometrische Verfahren:** Authentifikation von Personen auf Grund von unveränderlichen körperlichen Merkmalen, wie Stimme, Gesicht, Fingerabdruck,… .Es werden 2 Arten von Fehlern unterschieden.
  1. **Falsch positive Klassifikation**: es wird einem Unbefugten Zugang gewährt (falsche Akzeptanz, FAR = false acceptance rate)
  2. **Falsch negative Klassifikation**: es wird ein Befugter abgewiesen (falsche Zurückweisung, FRR = false rejection rate)

  Idealerweise sollte bei einem Authentifizierungssystem beide Arten von Fehlern sehr klein sein. Gute kryptographische Verfahren kommen sehr nahe, biometrische Verfahren sind sehr weit entfernt. Aus diesem Grund werden sie eine Authentifikation mittels starker Kryptographie in absehbarer Zeit nicht ersetzen, jedoch dienen sie als Ergänzung.

Kapitel 7 Public-Key-Infrastruktur

Schlüsseltauschproblem gelöst. Öffentliche Schlüssel werden heute öffentlich bereitgestellt, auf so genannten Key-Servern oder in den Trustcentern, wie bei einem Telefonbuch.

=> **Problem**: Angreifer kann durch einen Man-in-the-Middle-Angriff den öffentlichen Schlüssel von Alice abfangen und seinen öffentlichen Schlüssel im Namen von Alice publizieren.

=> **Lösung**: es muss jeder Benutzer des öffentlichen Schlüssels von Alice dessen Authentizität überprüfen

- **Persönliche Prüfung öffentlicher Schlüssel**: Alice könnte ihren Schlüssel persönlich auf einer Diskette an Bob übergeben, oder ihn anrufen. Dieses Verfahren ist jedoch viel zu aufwendig.

- **Trustcenter**: Die wichtigste Aufgabe des Trustcenters besteht in der Zertifizierung öffentlicher Schlüssel von Privatpersonen, Firmen oder anderen Institutionen. Es gibt Zertifikate verschiedener Klassen. Klasse 1 und 2 können nicht als sicher bezeichnet werden, da keine strenge Überprüfung der Personalien. Klasse 3 wird erteilt nach PostIdent-Verfahren. Bei Klasse 4 muss man persönlich beim Trustcenter erscheinen.

  Ablauf:

  1. Alice besorgt sich ein Zertifikat der Klasse 3 oder 4
  2. Bob besorgt sich ein Zertifikat der Klasse 3 oder 4
  3. Alice und Bob schicken sich gegenseitig ihre zertifizierten öffentlichen Schlüssel
  4. Alice überprüft das Zertifikat auf Bobs öffentlichem Schlüssel
  5. Bob überprüft das Zertifikat auf Alices öffentlichem Schlüssel
  6. Bob und Alice können sich nun geheime und signierte Nachrichten zukommen lassen

  Wichtig bei diesem Prozess ist die Echtheit des öffentlichen Schlüssels des Trustcenters. Trustcenter erzeugt auch einen Zeitstempel unter dem Dokument, durch Anhängen von codierter Uhrzeit und Datum an das Dokument und Signatur des (über eine Einweg-Hash-Funktion f komprimierten) Resultat durch das Trustcenter mit dem geheimen Schlüssel S_T. ((M, Uhrzeit, Datum), E_{ST}(f(M, Uhrzeit, Datum)))

- **Zertifikatshierarchie**: Jede Firma besitzt eine eigene Certification Authority (CA) welche in der Firma die Aufgabe der Schlüsselzertifizierung und Verwaltung übernimmt. Die Schlüssel von Alice und Bob werden von der firmeneigenen CA1 zertifiziert.

- **Web-of-Trust**: Als PGP zum Einsatz kam, gab es noch keine CAs oder Trustcenter. Es werden zwar auch öffentliche Schlüssel digital signiert, allerdings nicht von speziellen Institution, sondern von Privatpersonen.

  => Carol möchte eine vertrauliche E-Mail an Bob schicken. Zuerst besorgt sie sich seinen öffentlichen Schlüssel. Sie akzeptiert ihn ohne persönliche Prüfung der Authentizität, denn Bobs Schlüssel wurde von Alice signiert und Carol vertraut Alice (d.h. Carol akzeptiert jeden Schlüssel der von Alice signiert ist)
- **Zukunft**: Um dem einzelnen Anwender die Verwaltung öffentlicher Schlüssel abzunehmen, wird ein zentraler Verzeichnisdienst benötigt. Das Lightweight Directory Access Protocol (LDAP) ist ein Protokoll um E-Mail-Adressen, Nutzerdaten, Dienstzuordnungen etc. in einem Rechnernetz komfortabel verwalten.

  => wichtige Regeln für die Handhabung von öffentlichen Schlüsseln:
  - Benutzen sie niemals einen Schlüssel den sie per Mail erhalten haben oder den sie im Internet gefunden haben, solange er nicht von einer Person ihres Vertrauens signiert ist!
  - Signieren sie niemals einen Schlüssel, den sie nur per Mail erhalten haben oder den sie irgendwo im Internet gefunden haben, auch wenn er von einer Person ihres Vertrauens signiert ist. Verifizieren Sie persönlich oder telefonisch die Echtheit des Schlüssels, den sie signieren.
  - Vertrauen in einen Schlüssel bedeutet nicht automatisch Vertrauen in den Besitzer des Schlüssels!

Kapitel 8 Public-Key-Systeme

Heute werden zur Kommunikation generell Public-Key-Systeme verwendet. Da sie langsamer wie symmetrische Blockchiffren arbeiten, wird in der Praxis die Hybride Verschlüsselung verwendet. Alice generiert Sitzungsschlüssel k -> verschlüsselt M mit einem symmetrischen Verfahren -> verschlüsselt k mit Bobs öffentlichem Schlüssel P_B -> schickt beide Chiffretexte an Bob -> Bob dechiffriert sie nacheinander.

- **PGP**: Programm zur sicheren Kommunikation per E-Mail. PGP steht für pretty good privacy. Man kann Dateien verschlüsseln oder signieren und das Resultat dann als E-Mail verschicken. Version 7.0 bietet neben Verschlüsseln und Signieren folgende Funktionen:
  1. **IDS** (intrusion detection system): Abwehr bestimmter Netzwerkangriffe
  2. **VPN** (virtual private networking): sichere Kommunikation zwischen Firmenfilialen
  3. Automatische Ver- und Entschlüsselung von Dateien auf der Festplatte
  4. Personal-Firewall zur Filterung von ein- und ausgehenden Paketen
  5. Einfacher (automatischer) Zugang zu Trustcenter, CA und Schlüsselservern

  - **Schlüsseltausch mitPGP**: PGP-Benutzer besitzt auf seinem Rechner einen Schlüsselring mit den Schlüsseln aller Kommunikationspartner, mit beliebig vielen Signaturen. PGP konnte nicht auf eine zentrale Instanz vertrauen (Trustcenter), daher Web-of-Trust.

  - **Die Big-Brother-Funktion**: jede verschlüsselte Nachricht wird nicht nur mit dem Public-Key des Empfängers verschlüsselt, sondern auch mit dem Firmen-Public-Key. Dieser wird als CMRK (Corporate-Message-Recovery-Key) oder ADK (Additional Key) bezeichnet. Firmenchef kann alle E-Mails mitlesen.
    Kann sehr hilfreich sein, muss aber firmenintern geregelt werden, wann er benutzt werden darf und wann nicht.

  - **GnuPG (Gnu-Privacy-Guard):** PGP-Variante ohne Gebühren. Symmetrische Algorithmen wie AES, triple-DES und TWOFISH werden verwendet. RSA, ElGamal,DAS sind die unterstützten Public-Key-Verfahren. Hält sich an den OpenPGP-Standard.

  - **Angriffe gegen PGP**: manipulierter Schlüssel: Mallory hat Zugang zum gespeicherten geheimen Schlüssel, kann er diesen einfach in Erfahrung bringen, ohne das Passwort (passphrase) mit dem der geheime Schlüssel verschlüsselt ist.
    => **Ablauf**: Mallory verändert zufällig einige Bits des gespeicherten, verschlüsselten geheimen Schlüssels von Alice und bringt Alice dazu, eine ihm bekannte Nachricht M zu signieren. Aus der Signatur S und der Nachricht M kann er nun den Schlüssel ermitteln.
    Der Angriff ist sowohl bei DAS-Signaturen möglich, als auch bei RSA-Signaturen, sofern der geheime Schlüssel nach dem OpenPGP-Standard gespeicher wird.

- **S/MIME und das X.509 Protokoll**: steht für Secure Multipurpose Internet Mail Extension und ist ein Protokoll zum Verschlüsseln und Signieren von E-Mail. Benutzt Standard PKCS #7 (Cryptographic Message Syntax) für verschlüsselte E-Mails und X.509 für Zertifikate. (PKCS = public key cryptography system).
  Ein X.509 Zertifikat besteh aus folgenden Teilen:
  1. **Version**
  2. **Seriennummer**: innerhalb einer CA eindeutig
  3. **Algorithmenidentifikation**: Spezifikation des verwendeten Signaturalgorithmus und der Einweg-Hash-Funktion
  4. **Aussteller**: Identifikation der ausstellenden CA
  5. **Geltungsdauer**
  6. **Subject**: Person oder Objekt, deren/dessen Identität das Zertifikat sichern soll
  7. **Public-Key**: Öffentlicher Schlüssel des Subject. Namen des Algorithmus, Schlüssellängem den Modul n und den öffentlichen Exponenten e
  8. **X.509v3-Erweiterungen**: Raum für spezielle Parameter der jeweiligen Anwendung
  9. **Signatur**: Signatur des Hash-Werts aller anderen Felder durch den Aussteller
     => untrennbare Verbindung zwischen dem Namen und dem öffentlichen Schlüssel einer Person

- **OpenPGP versus S/MIME**: S/MIME hat sich in der Industrie durchgesetzt und PGP bei den Benutzern. Es wird versucht diese beiden Standards zusammen zu führen.

- **Secure shell (SSH**): wird benutzt um über lokale Netze auf einen entfernten Rechner (Server) zu arbeiten.
  => **Probleme**: Hacker kann das vom Server angeforderte und im Klartext übertragene Passwort einfach abhören, außerdem kann der Hacker alle Daten die unverschlüsselt übertragen werden abhören.
  => SSH bietet Abhilfe durch Einsatz von Public-Key-Verfahren
  SSH bietet:
  1. Sichere gegenseitige Authentifizierung von Client und Server
  2. Sicheres Login durch Authentifikation der Benutzer auf dem Server. Durch RSA oder DSA wird Identität geprüft
  3. Kommunikation ist verschlüsselt. Mit RSA wird Sitzungsschlüssel vereinbart, der dann zur symmetrischen Verschlüsselung mit IDEA, Blowfish oder triple-DES dient
  4. Sichere Verbindung auch für grafische Anwendungen
  5. IP-Tunneling ist möglich => ftp-Verbindung kann abgesichert werden

- **Secure socket layer (SSL):** ursprünglich nur für die Absicherung von http-Verbindungen zwischen Web-Browser und Server gedacht. Version 3 dient als Grundlage für eine Spezifikation der Transport Layer Security (TLS). Zwischen Transport- und Anwendungsschicht.
  Die Protokolle erfüllen folgende Aufgaben:
  1. **SSL-Record-Protokoll**: kryptographische Algorithmen für Verschlüsselung und Authentifikation werden bereitgestellt. Es wird MD5 oder SHA-1 für die Einweg-Hash-Funktion benutzt. Für die Authentifizierung wird RSA oder Diffie-Hellmann verwendet und für symmetrische Verschlüsselung stehen RC4, RC2, DES, DES40, Triple-DES, IDEA und Fortezza bereit.
  2. **SSL-Alert-Protokoll**: Warn- und Fehlermeldungen
  3. **SSL-Change-Cipher-Protokoll**: starten der Initialisierung der ausgewählten Algorithmen
  4. **SSL-Handshake-Protokoll**: Authentifikation von Client und Server durch Übertragung der entsprechenden Zertifikate. Parallel erfolgt der Schlüsseltausch für die symmetrische Verschlüsselung
  5. **http**: besorgt die Übertragung der HTML-Seiten vom Server zum Client

  Wichtig sind die Rechnerzertifikate, insbesondere beim Server. Dies sind von der CA signierte öffentliche Schlüssel von Server und Client. Damit Client sicher ist, dass er auf der richtigen Seite ist, muss er das Zertifikat des Server-Schlüssels überprüfen. Ist der Server-Schlüssel nicht von einer Client bekannten CA signiert, so wird Benutzer gewarnt und gefragt ob er ihn trotzdem akzeptieren will.

- **Virtual Private Networking und IP Security**: Verschlüsselung und Authentisierung soll funktionieren ohne dass Benutzer etwas davon bemerkt.
  =>**VPN** (virtual private networking): Für einfache Anwendungen kann schon mit SSH oder SSL ein VPN aufgebaut werden. Jedoch nicht ausreichend.
  =>**IP Security** (IPSec): Absicherung von Internet-Paketen.
  => **Vorteile**:
  1. Starke kryptographische Sicherheit für Verkehr nach außen ohne Belastung des internen Verkehrs
  2. IPSec auf einem Firewall kann nicht umgangen werden
  3. IPSec arbeitet unter Transportschicht -> die Anwendungssoftware muss nicht geändert werden
  4. Arbeit von IPSec wird nicht bemerkt.